BOSTON UNIVERSITY

Questrom School of Business

Name: Zhankai Zhang
BU ID: U38625790
Email: zzk2001@bu.edu
Course: Advanced Machine Learning (MF815)
Project: Company Bankruptcy Prediction
Date: 03/18/2026

Abstract

This project develops a corporate bankruptcy prediction model using 95 financial ratio features from the Taiwan Economic Journal, addressing an extremely imbalanced dataset in which only 3% of the 5,455 training samples represent bankrupt firms. Five classification methods—Random Forest, XGBoost, lightGBM, catboost, and Logistic Regression—were systematically evaluated, with Bayesian hyperparameter optimization conducted via a 200-trial **Optuna** search. **XGBoost** achieved the strongest baseline performance, with AUC 0.9625 and competitive minority-class F1. After incorporating an **ECOD**-based outlier detection feature, the tuned XGBoost model improved the validation F1 to **0.5060**, demonstrating enhanced capability in identifying rare default cases.

Although additional techniques such as **SMOTE** oversampling, **Focal Loss**, model **stacking** (XGBoost + LightGBM + CatBoost), and probability averaging were explored, none delivered further gains, largely due to overfitting or limited model diversity. The final submission employed full-data retraining, removal of highly collinear features, and multi-seed prediction averaging, achieving an OOF F1 of **0.4988** and predicting a 4.03% bankruptcy rate on the test set. These results highlight the importance of imbalance-aware optimization and targeted feature engineering in rare-event financial risk modeling.

1. Methods

1.1 Model Selection Logic – XGBoost, Random Forest, LightGBM, Logitstic Regression

The selection of the three models is primarily motivated by findings from the EDA. Specifically, the EDA revealed three key characteristics of the dataset: **nonlinear relationships, extreme outliers, and severe class imbalance**. These characteristics correspond to the strengths of the selected models: **XGBoost** (nonlinearity and imbalance handling), **Random Forest** (robustness and bagging diversity), **LightGBM**, and **Logistic Regression** (linear baseline for comparison).

The choice of these models was largely informed by the EDA results. First, the Predictive Power Score (PPS) was used to detect nonlinear predictive relationships that are not captured by Pearson correlation (Wetschoreck et al., 2020). The analysis revealed that several features exhibit nonlinear predictive signals with respect to the bankruptcy outcome. For example, Net Value Growth Rate shows a PPS of 0.11 while its Pearson correlation is approximately zero (ignore the Unnamed, I drop this col in the after), indicating the presence of nonlinear relationships that require nonlinear modeling approaches. This observation motivated the use of **tree-based models**.

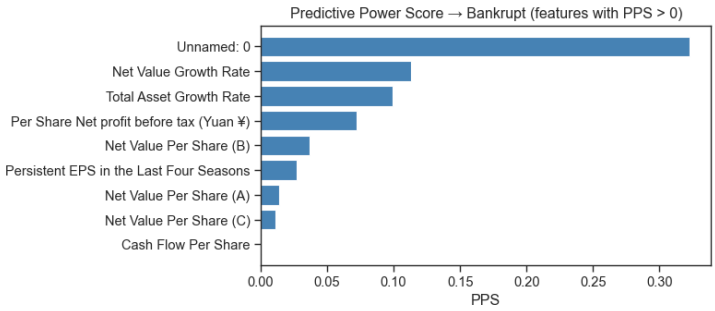


Figure 1: Predictive Power Score

Another important characteristic of the dataset is the **severe class imbalance** in the target variable, with a 97/3 distribution between non-bankrupt and bankrupt firms. Tree-based models provide several mechanisms to address this issue. For instance, **Random Forest** can handle imbalance through class_weight='balanced_subsample' (Kuhn & Johnson, Ch.16), while **XGBoost** incorporates imbalance directly in the loss function via the scale_pos_weight parameter. Similarly, **LightGBM** offers the is_unbalance=True option to automatically adjust for the 97/3 imbalance. In addition, other imbalance-handling techniques such as **SMOTE** and **threshold adjustment** were also explored and are discussed later in the paper.

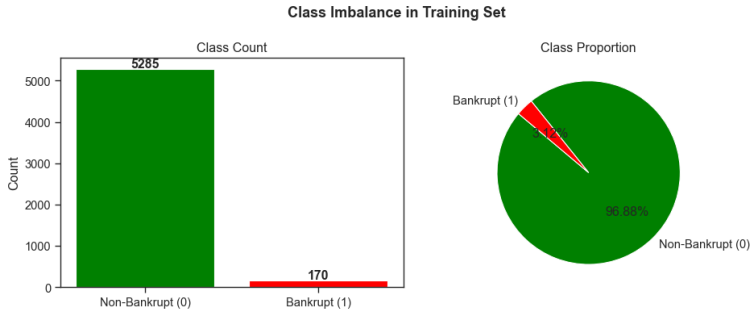


Figure 2: Target Analysis

The dataset also contains 21 columns with **missing values**, resulting from converting certain **sentinel values** to NaN during preprocessing, as well as **extreme outliers**. **XGBoost** natively supports missing values during tree construction, while **Random Forest** is inherently robust to outliers because tree-based splits are generally unaffected by extreme values.

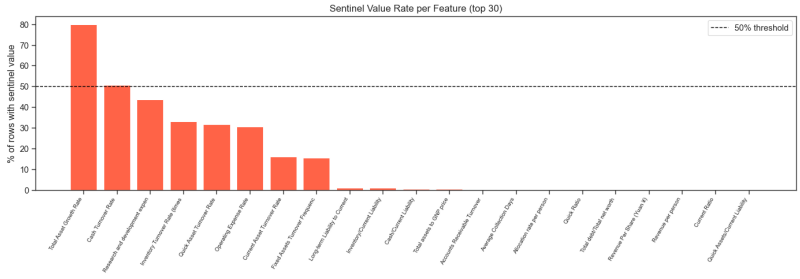


Figure 3: Sentinel Value Analysis

Another issue identified in the dataset is **multicollinearity**. In **XGBoost**, the combination of **L1/L2 regularization** and feature subsampling (colsample_bytree) helps mitigate the impact of redundant features (for **Logistic Regression**, highly collinear variables were removed during preprocessing). This is particularly relevant because the dataset contains **over 90 features**, many of which are highly correlated **financial ratios**.

Finally, **LightGBM** was also considered due to **its leaf-wise tree growth strategy**, which is generally more efficient than the level-wise strategy used by XGBoost in identifying informative splits. Given the dataset size (5,455 observations with more than 90 features), LightGBM's num_leaves parameter provides more flexible control of model complexity compared with max_depth, allowing the model to efficiently capture complex feature interactions.

1.2 Detailed Preprocessing and Modeling Techniques *(Ordered by Estimated Impact)*

1.2.1 Class Imbalance Handling

Threshold Optimization via the Precision–Recall Curve: Instead of using the default classification threshold of 0.5, the optimal decision threshold was determined by searching for the probability cutoff that maximizes the F1 score on the validation set. Because the F1 score balances precision and recall, this threshold adjustment is particularly important in highly imbalanced classification problems where the positive class is rare. In practice, this post-processing step substantially improved model performance, increasing the F1 score by 10–20 percentage points compared with the default threshold.

Class Weighting / Cost-Sensitive Training: Random Forest: class_weight='balanced'; **XGBoost:** scale_pos_weight = 31 (n_negative / n_positive ≈ 5285/170 ≈ 31); **LightGBM:** is_unbalance=True These approaches modify the effective contribution of minority-class samples during training, encouraging the model to pay greater attention to bankruptcy observations.

SMOTE (Synthetic Minority Over-sampling Technique) generates synthetic minority-class samples through k-nearest neighbor interpolation (k = 5) between existing minority observations (Chawla et al., 2002). To avoid data leakage, SMOTE was applied only within the training folds of cross-validation. Specifically, in the 5-fold cross-validation pipeline, I use oversampling only on the four training folds, while the validation fold remained untouched.

SMOTE-ENN. After synthetic samples are generated using SMOTE, Edited Nearest Neighbors (ENN) is applied to remove noisy or ambiguous observations near class boundaries (Chawla et al., 2002; Wilson, 1972).

SMOTE-Tomek Links. This method first performs SMOTE oversampling and then removes Tomek links, which are pairs of observations from different classes that are each other's nearest neighbors. Removing such pairs helps clean overlapping class boundaries (Chawla et al., 2002; Tomek, 1976).

In practice, however, none of these methods improved the final results. As noted by Kuhn & Johnson (2013), SMOTE can inflate cross-validation performance but may fail to generalize to truly unseen data. Synthetic samples generated by SMOTE interpolate between existing minority-class observations and fill the feature space around them. During cross-validation, these synthetic points can introduce similarity between training and validation folds, leading to overly optimistic F1 estimates. When evaluated on an independent validation set, this advantage often disappears, while the precision penalty remains.

Model	CV F1	Val F1
XGBoost (baseline)	0.4715	0.4255
SMOTE + XGBoost	0.4773	0.3889
SMOTEENN + XGBoost	0.4726	0.3925
XGBoost (top-30)	0.4451	0.4082
SMOTE + XGBoost (top-30)	0.4726	0.3883
SMOTEENN + XGBoost (top-30)	0.4171	0.3765

Figure 4: Comparison between different combination

1.2.2 ECOD (Empirical Cumulative Distribution Function Outlier Detection): ECOD was used as an unsupervised anomaly detection method to identify observations that deviate significantly from the overall distribution. ECOD computes outlier scores based on empirical cumulative distribution functions for each feature, allowing the detection of extreme observations without strong distributional assumptions (Zhao et al., 2020). The resulting anomaly score was converted into a binary feature (Is_outlier), indicating whether a sample is flagged as an outlier.

Empirical analysis showed that bankrupt firms were approximately 5.7 times more likely to be flagged as outliers than non-bankrupt firms, suggesting that abnormal financial ratios are strongly associated with bankruptcy events. Incorporating this engineered feature improved model performance, increasing the **validation F1 score from 0.4706 to 0.5060.**

1.2.3 Missing Value Flag Engineering: Binary missing-value indicator variables were engineered. For each feature with a missing rate greater than 5%, an additional binary variable was created to indicate whether the original value was missing. This step was applied only when the bankruptcy rate among missing observations was at least 1.2 times higher than that of non-missing observations, suggesting that the missingness was potentially informative rather than random. **This approach allows models—especially tree-based models—to capture missingness patterns as predictive signals**

1.2.4 Other Data Processing and Modeling Techniques

- **Missing Value Analysis:** Columns with more than 50% missing values were removed.
- **Median Imputation:** Missing values were filled using SimpleImputer (strategy='median').
- **StandardScaler Normalization:** Z-score applied primarily for Logistic Regression.

- **Yeo-Johnson Power Transformation:** Power transformation applied to continuous features to reduce skewness and approximate normal distributions; binary indicator variables were excluded, and the transformation was fitted only on the training data (Yeo & Johnson, 2000).
- **Feature Subset Selection:** STRONG_FEATURES (21 features) and ALL_FEATURES (94 features), primarily evaluated in Random Forest experiments.
- **RFECV (Recursive Feature Elimination with Cross-Validation):** Iteratively removed the least important features (step = 5) using 5-fold cross-validation with F1 as the evaluation metric to determine the optimal feature subset. This approach showed limited improvement for tree-based models (Guyon et al., 2002).
- **Focal Loss:** This loss function reduces the weight of easy negative samples and forces the model to focus on difficult positive samples. Implemented in XGBoost through custom gradient and Hessian functions (Lin et al., 2017).
- **OOF Stacking (Out-of-Fold Stacking):** First-layer models (XGBoost, LightGBM, CatBoost) generated out-of-fold probability predictions using 5-fold StratifiedKFold, and Logistic Regression was used as the second-layer meta-learner to combine predictions.
- **Multi-Seed Averaging:** The same model was trained with 10 different random seeds, and predictions were averaged to reduce variance and improve stability.

2. Hyperparameter Tuning: Optuna Bayesian Optimization

All models were tuned using **Optuna** with the **Tree-structured Parzen Estimator (TPE)** sampler. Unlike exhaustive grid search or uniform random search, the TPE algorithm models the conditional probability ($P(hyperparameters \mid score)$) and iteratively proposes more promising configurations based on the outcomes of previous trials (Bergstra et al., 2011). This Bayesian optimization framework allows the search process to concentrate on regions of the hyperparameter space that are more likely to yield improved performance, making it substantially more **sample-efficient** and capable of identifying better solutions with fewer evaluations.

The tuning objective was to **maximize the F1-score of the minority class (bankruptcy)** using 5-fold Stratified Cross-Validation**.** Different sampling distributions were applied depending on parameter characteristics: **log-uniform sampling** was used for scale-sensitive parameters such as learning rates and regularization terms, while uniform sampling was used for tree structure parameters.

The total search budget was allocated based on model complexity: **XGBoost:** 200 trials; **LightGBM:** 200 trials; **CatBoost:** 100 trials; **Random Forest:** 50 trials

Model	Trials	CV F1 (before)	CV F1 (after)	Val F1 (before)	Val F1 (after)	AUC-ROC
XGBoost	200	0.3717	0.5435	0.4255	**0.4706**	0.9618
LightGBM	200	0.3560	0.5882	0.4324	0.4483	0.9605
Random Forest	50	0.4158	0.5329	0.4158	0.3944	0.9489
CatBoost	100	—	—	—	0.4286	—
Logistic Regression	—	—	—	0.4337	0.4337	0.9155

Table 1: Tuning Results Comparison

XGBoost achieved the best validation performance after tuning, with a Val F1 score of 0.4706, representing approximately a **10.6%** improvement over the baseline model**.**

LightGBM achieved the highest cross-validation F1 score (**0.5882**), but its lower validation F1 (**0.4483**) suggests a degree of overfitting during cross-validation**.**

For **Random Forest**, validation performance slightly declined after tuning (**0.4158 → 0.3944**), indicating that the baseline configuration with class_weight='balanced' was already close to the optimal setting.

Logistic Regression was not extensively tuned. In this model, performance improvements are primarily driven by threshold optimization rather than hyperparameter adjustments**,** since the model contains relatively few tunable parameters.

3. Results

	Model Type	Approach	Model	Accuracy	F1 (Class 0)	F1 (Class 1)	AUC-ROC	Threshold
0	XGBoost	Tuned + ECOD	XGB (tuned + ECOD)	0.9624	0.9805	0.5060	0.9627	0.4972
1	XGBoost	Tuned + Yeo-Johnson	XGB (tuned + YJ)	0.9633	0.9810	0.4872	0.9619	0.6267
2	XGBoost	Optuna Tuned	XGB (tuned)	0.9588	0.9785	0.4706	0.9618	0.4744
3	XGBoost	Re-tuned (v2)	XGB v2 (f1_best_threshold)	0.9606	0.9795	0.4691	0.9625	0.6244
4	XGBoost	Tuned + RFECV	XGB (tuned + RFECV)	0.9588	0.9786	0.4578	0.9629	0.5678
5	LightGBM	Optuna Tuned	LGBM (tuned)	0.9707	0.9849	0.4483	0.9605	0.7259
6	Logistic Reg.	Baseline	LR (all features)	0.9569	0.9776	0.4337	0.9155	0.9171
7	XGBoost	Tuned + ECOD + SMOTE	XGB + SMOTE-ENN	0.9496	0.9736	0.4330	0.9558	0.9627
8	LightGBM	Baseline	LGBM (all features)	0.9615	0.9801	0.4324	0.9578	0.2329
9	CatBoost	Optuna Tuned	CatBoost (tuned)	0.9633	0.9811	0.4286	0.9638	0.6370
10	XGBoost	Baseline	XGB (all features)	0.9505	0.9741	0.4255	0.9625	0.0675
11	XGBoost	Tuned + Focal Loss	XGB (tuned + Focal)	0.9624	0.9806	0.4225	NaN	0.2951
12	XGBoost	Tuned + ECOD + SMOTE	XGB + SMOTE-Tomek	0.9496	0.9736	0.4211	0.9529	0.8955
13	Random Forest	Baseline	RF (all features)	0.9615	0.9801	0.4167	0.9487	0.2467
14	Random Forest	Baseline	RF (strong features)	0.9459	0.9716	0.4158	0.9210	0.1267
15	XGBoost	Tuned + ECOD + SMOTE	XGB + SMOTE	0.9533	0.9757	0.4138	0.9526	0.9273
16	XGBoost	Baseline (top30)	XGB (top 30 features)	0.9468	0.9722	0.4082	0.9588	0.0645
17	Random Forest	Baseline + SMOTE	RF + SMOTE (all)	0.9624	0.9806	0.4058	0.9432	0.2600
18	Random Forest	Baseline + SMOTE	RF + SMOTE (strong)	0.9423	0.9697	0.4000	0.9248	0.1200
19	Logistic Reg.	Baseline	LR (strong features)	0.9533	0.9757	0.4000	0.9413	0.8727
20	Random Forest	Optuna Tuned	RF (tuned)	0.9606	0.9796	0.3944	0.9489	0.5593
21	Random Forest	Baseline + SMOTE	SMOTE + RF (all)	0.9322	0.9641	0.3934	0.9513	0.3467
22	XGBoost	Baseline + SMOTE	XGB + SMOTE-ENN (all)	0.9404	0.9687	0.3925	0.9506	0.7591
23	XGBoost	Baseline + SMOTE	XGB + SMOTE (all)	0.9597	0.9791	0.3889	0.9541	0.9254
24	XGBoost	Baseline + SMOTE (top30)	XGB + SMOTE (top30)	0.9423	0.9697	0.3883	0.9508	0.3893
25	XGBoost	Baseline + SMOTE (top30)	XGB + SMOTE-ENN (top30)	0.9514	0.9747	0.3765	0.9500	0.9970
26	LightGBM	Baseline	LGBM (strong features)	0.9450	0.9712	0.3750	0.9211	0.1075
27	XGBoost	Baseline	XGB (strong features)	0.9597	0.9792	0.3714	0.9222	0.3269
28	Random Forest	Baseline + SMOTE	SMOTE + RF (strong)	0.9368	0.9667	0.3551	0.9316	0.4067

Figure 5: All model results

	Model Type	Model	Accuracy	F1 (Class 0)	F1 (Class 1)	AUC-ROC
0	XGBoost	XGB (tuned + ECOD)	0.9624	0.9805	0.5060	0.9627
1	LightGBM	LGBM (tuned)	0.9707	0.9849	0.4483	0.9605
2	Logistic Reg.	LR (all features)	0.9569	0.9776	0.4337	0.9155
3	CatBoost	CatBoost (tuned)	0.9633	0.9811	0.4286	0.9638
4	Random Forest	RF (all features)	0.9615	0.9801	0.4167	0.9487

Figure 6: Best model per type

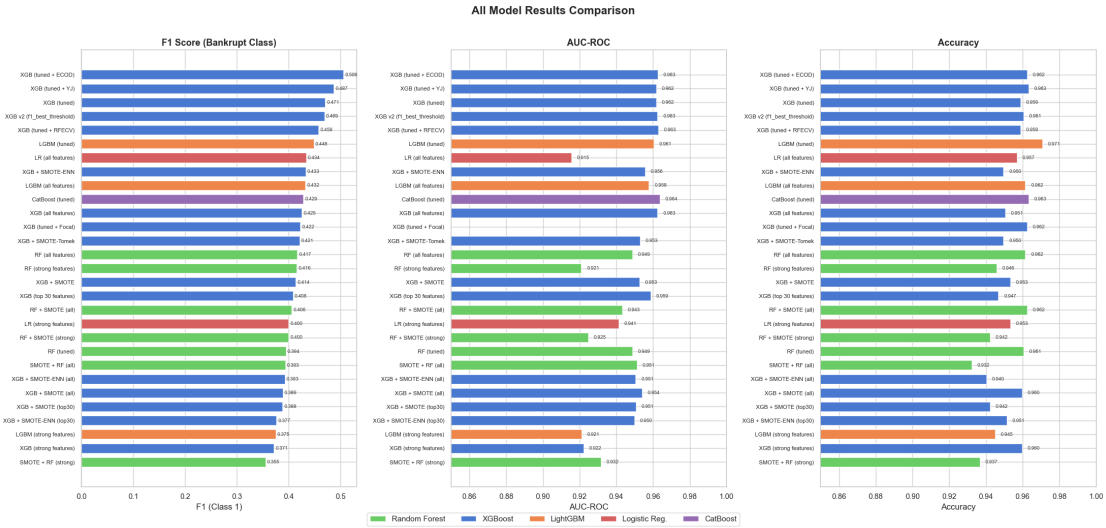


Figure 7: All model results comparison

4. Best Model Selection: XGBoost (tuned + ECOD), Val F1 = 0.5060

The modeling process followed a sequential workflow beginning with baseline model comparison. Initial experiments included **Random Forest**, **LightGBM**, **XGBoost**, and **Logistic Regression**. Following the recommendation from Applied Predictive Modeling by Max Kuhn and Kjell Johnson (Chapter 16), model comparison was first conducted using AUC-ROC, after which the classification threshold was optimized to maximize the F1 score for the minority class. Among the baseline models, XGBoost achieved the highest AUC-ROC (0.9601) while maintaining a strong validation F1 score (0.4255). Based on this result, XGBoost was selected as the primary baseline model for subsequent experimentation.

Several imbalance-handling and feature selection methods were then tested, including multiple **SMOTE-based resampling strategies** and feature subset experiments. However, none of these approaches improved validation performance relative to the baseline (see the *Imbalance Handling* section for details). Consequently, the baseline XGBoost model was retained as the foundation for further optimization.

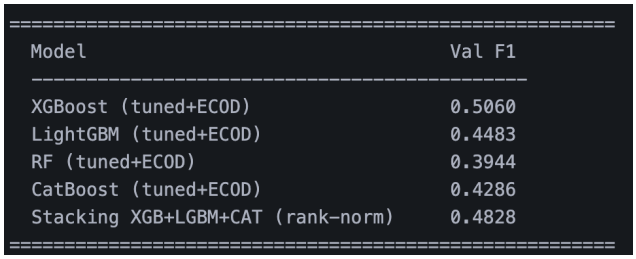
Next, hyperparameter tuning was performed using **Optuna** with the **TPE sampler**, resulting in **XGBoost (tuned)** with a **Val F1 of 0.4706**. Building on this tuned model, several additional techniques were evaluated, including **ECOD outlier detection**, **Yeo–Johnson transformation**, **RFECV**, and **Focal Loss**. Among these methods, **ECOD feature engineering** produced the largest improvement. Adding the ECOD-based outlier indicator increased the **Val F1 to 0.5060**, yielding the best-performing single model: **XGBoost (tuned + ECOD)**.

After identifying the best individual model, a stacking ensemble architecture was also explored to investigate whether combining multiple models could further improve predictive performance. To ensure fairness among base learners, all models were retrained on the same feature space, including the ECOD-generated outlier feature. Since ECOD produced the largest single improvement for XGBoost, the same feature set was applied to the other models as well.

Specifically, LightGBM (200 trials), Random Forest (50 trials), and CatBoost (100 trials) were re-tuned using Optuna with the TPE sampler and 5-fold stratified cross-validation on the ECOD-augmented training dataset (91 features).

The stacking architecture was defined as follows: First Layer (Base Learners): XGBoost, LightGBM, CatBoost. Each base learner generated out-of-fold (OOF) probability predictions using 5-fold stratified cross-validation. Second Layer (Meta-Learner): Logistic Regression.

Before training the meta-learner, the OOF probabilities were transformed using rank normalization to remove scale differences between model probability outputs.



Model	Val F1
XGBoost (tuned+ECOD)	0.5060
LightGBM (tuned+ECOD)	0.4483
RF (tuned+ECOD)	0.3944
CatBoost (tuned+ECOD)	0.4286
Stacking XGB+LGBM+CAT (rank-norm)	0.4828

Figure 8: Stacking models

Despite the additional stacking experiments, the ensemble model did not outperform the best individual model. The **XGBoost (tuned + ECOD)** configuration remained the strongest performer, achieving **the highest validation F1 score of 0.5060**. Therefore, XGBoost (tuned + ECOD) was selected as the final model for bankruptcy prediction.

5. AI Disclosure:

Claude Code (by Anthropic) was used for code assistance, debugging, and exploratory analysis during this project. Specifically, I use the /init function at the beginning of the development process to provide the system with project context, including assignment background, feature descriptions from Kaggle, and relevant literature on handling imbalanced datasets. This allowed the assistant to provide more context-aware technical suggestions.

Claude Code was also used to generate the initial project structure and code scaffolding. For certain implementation details—such as ECOD outlier detection, SMOTE-based resampling, Predictive Power Score (PPS) analysis, and probability threshold optimization—the methods were implemented by the author after reviewing the relevant academic and technical references.

In some cases, Claude Code suggested additional techniques, including missing-value indicator engineering, stacking ensemble models, and the use of CatBoost as an alternative gradient boosting algorithm. These suggestions were evaluated and selectively incorporated into the modeling experiments.

All model selection decisions, experimental design, result interpretation, and report writing were conducted by the author.

References

Bergstra, J., Bardenet, R., Bengio, Y., & Kégl, B. (2011). Algorithms for hyper-parameter optimization. Advances in Neural Information Processing Systems (NeurIPS), 24.

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. Journal of Artificial Intelligence Research, 16, 321–357. https://doi.org/10.1613/jair.953

Chen, T. & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *KDD*.

Guyon, I., Weston, J., Barnhill, S., & Vapnik, V. (2002).Gene selection for cancer classification using support vector machines.Machine Learning, 46(1–3), 389–422.

Kuhn, M. & Johnson, K. (2013). *Applied Predictive Modeling*, Ch. 16: Remedies for Severe Class Imbalance.

Lin, T. Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal loss for dense object detection.Proceedings of the IEEE International Conference on Computer Vision (ICCV).

Tomek, I. (1976). Two modifications of CNN. IEEE Transactions on Systems, Man, and Cybernetics, 6, 769–772.

Wilson, D. L. (1972). Asymptotic properties of nearest neighbor rules using edited data. IEEE Transactions on Systems, Man, and Cybernetics, 2(3), 408–421.

Wetschoreck, F., Krabel, T., & Thiele, M. (2020). Predictive Power Score: A universal measure for predictive relationships. GitHub Repository. https://github.com/8080labs/ppscore

Yeo, I. K., & Johnson, R. A. (2000).A new family of power transformations to improve normality or symmetry.Biometrika, 87(4), 954–959.

Zhao, Z., Nasrullah, Z., & Li, Z. (2020). ECOD: Unsupervised outlier detection using empirical cumulative distribution functions. IEEE International Conference on Data Mining (ICDM).